Final Report

Jiahui Cheng and Hojin Chang

March, 2026

1 Introduction

Assigned with the task of improving the IPC of several benchmarks, we began with a characterization study of four target workloads—NQueens, Coin, QuickSort, and Esift2—running on the in-order baseline processor. The baseline profiling revealed the following performance summary:

Metric	**NQueens**	**Coin**	**QuickSort**	**Esift2**
Baseline CPI	13.63	1.20	2.37	2.80
Baseline IPC	0.073	0.836	0.422	0.357
#1 Bottleneck	I$ miss/flush	EX overload	D$ miss	D$ miss

Table 1: Baseline processor performance and dominant bottleneck per benchmark.

The bottlenecks and emphasis differ across benchmarks: NQueens exhibits the worst CPI by far, dominated by instruction-cache misses and pipeline flushes; Coin is limited by execute-stage overload; while QuickSort and Esift2 are both data-cache–miss–bound. Taking into account that NQueens has the worst CPI and represents the largest optimization opportunity, we decided to address its bottleneck first.

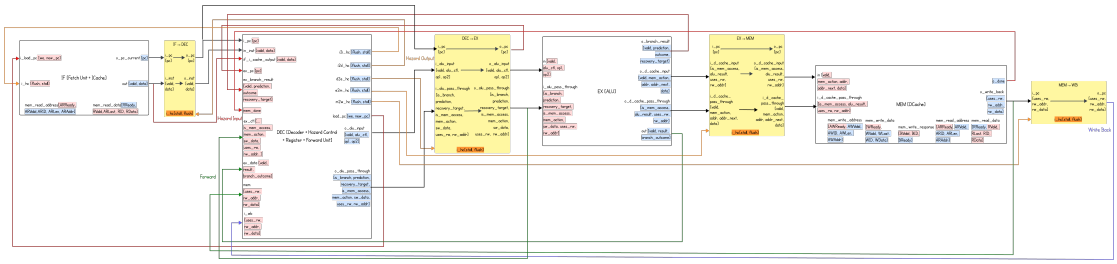


Figure 1: Baseline Processor Top-level Diagram

Modern high-performance processors extensively exploit instruction-level parallelism (ILP) through two complementary techniques: out-of-order execution, which dynamically reorders instructions to hide latencies, and superscalar issue, which dispatches multiple instructions per cycle. Given the benchmark characterization, it seemed a natural starting point to change the architecture to be both superscalar and out-of-order. In this project we combine both techniques in a MIPS processor that fetches, decodes, renames, and commits up to two instructions per cycle while executing them out of program order.

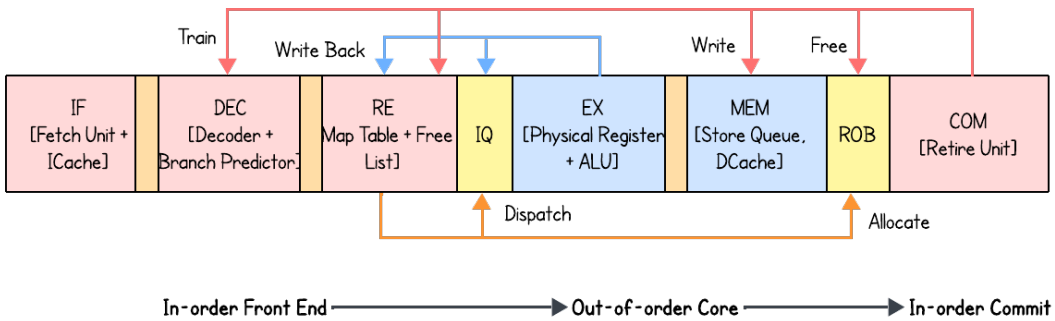
We considered the possibility that out-of-order execution and superscalar issue might not be universally beneficial—for instance, a larger branch mispredict penalty paired with a poor predictor

could negate the gains, and memory-bound workloads might not benefit from additional execution bandwidth, especially given that our simulated memory latency exceeds 100 cycles. After the architectural change to OoO superscalar, we therefore added fine-tuned module-level optimizations to address the specific problems exhibited by each benchmark. These optimizations span branch prediction, instruction cache design, and data cache design.

2 Design

2.1 Microarchitecture Overview

The processor is organized into seven logical stages connected by pipeline registers and decoupled by the Issue Queue (IQ) and the Reorder Buffer (ROB).



1. The front-end stages (IF → DEC → RE) are in-order and process up to two instructions per cycle.
2. The Issue Queue decouples the in-order front end from the out-of-order execution back end (Execute and Mem stage).
3. The Commit stage retires up to two instructions per cycle from the ROB head, maintaining precise architectural state.

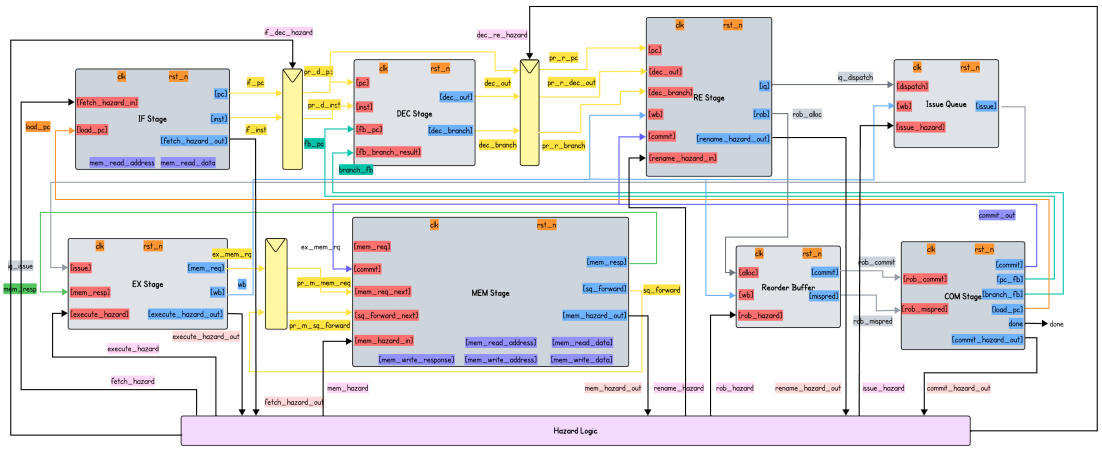


Figure 2: Superscalar Out-of-Order Processor Top-level Diagram

2.2 Pipeline Stage Details

2.2.1 Instruction Fetch (IF)

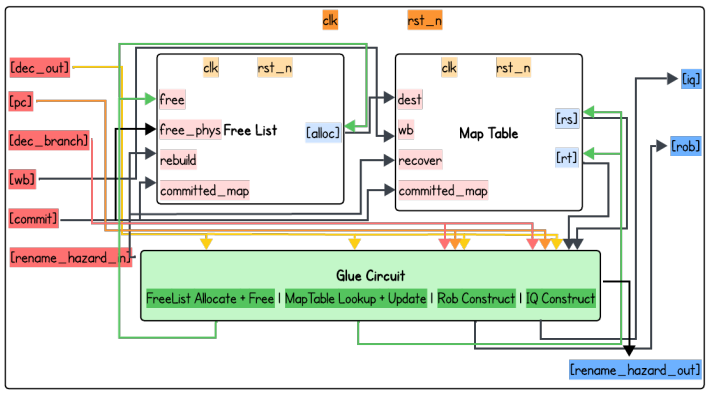
The Fetch Unit maintains the program counter and drives the I-Cache with two consecutive word addresses per cycle (PC and PC+4), enabling dual-issue of sequential instructions within the same cache line. A **pre-decoder** inspects the opcode of each fetched word to detect branch/jump instructions.

- If slot 1 contains a branch, it is suppressed and the cycle falls back to single-issue, ensuring that only slot 0 interfaces with the Branch Controller.
- When both slots fall at the end of a cache line, single-issue is also used since the second word would cross a line boundary.

2.2.2 Instruction Decode (DEC)

Two parallel decoder instances operate on both fetched slots simultaneously. Each decoder extracts the opcode, function code, register addresses, immediate values, and ALU control signals. Branch prediction, Return Address Stack (RAS) push/pop, and redirect logic operate only on slot 0's instruction, since slot 1 is guaranteed to be a non-branch by the pre-decoder. The branch predictor sits in this stage, providing zero-penalty correct predictions (the delay slot executes unconditionally in the next cycle).

2.2.3 Rename (RE)



The Rename stage maps architectural register names to physical registers using three submodules:

- Map Table: A speculative mapping from 32 architectural registers to physical registers is maintained. Each entry also carries a busy bit, cleared when the corresponding writeback broadcast arrives. For dual-rename, slot 1's lookup is forwarded from slot 0's destination if a RAW dependency exists within the same rename group. On a pipeline flush, the map table is restored from the committed map snapshot held in the Retire Unit.
- Free List: A circular FIFO of available physical registers supplies new destinations. It supports allocating up to two registers per cycle (one per slot) and freeing up to two per cycle

(from commit). On flush, the free list is rebuilt by scanning the committed map and marking every physical register not present as free.

- ROB and IQ Allocation: Each renamed instruction is simultaneously allocated an ROB entry (with branch metadata, PC, old physical destination for recovery) and an IQ entry (with physical source/destination tags, ready bits, and ALU control).

2.2.4 Issue (IS)

The Issue Queue is a fully-associative buffer of `IQ_SIZE` entries. Each cycle, combinational logic scans all entries and selects up to two ready instructions:

- **Slot 0**: the oldest ready entry (any type), where "oldest" is determined by ROB index comparison with a phase bit to handle circular wrap-around. Memory-access instructions must additionally be the oldest memory instruction in the IQ to preserve in-order load/store issue.
- **Slot 1**: the oldest ready *non-memory* entry that has no RAW hazard against slot 0's destination.

Dispatch-time wakeup is implemented: if a writeback broadcast arrives in the same cycle as a dispatch, the incoming entry's ready bits are updated before insertion, avoiding a one-cycle bubble.

2.2.5 Execute (EX)

Two identical combinational ALUs execute the issued instructions in parallel. Operands are read from the centralized Physical Register File (PRF), which has four read ports (rs and rt for each slot) and two write ports. The PRF uses write-port priority (port 0 wins on collision) to handle the rare case of simultaneous writes to the same register. When slot 0 holds a load or store, the ALU computes the effective address, and the result is forwarded as a memory request to the Memory Stage. A structural hazard arises when a load response returns on the same cycle as an ALU instruction is being issued: the load response is given priority on write-back port 0, and the ALU instruction on slot 0 is replayed by asserting `execute_hazard_out.flush`.

2.2.6 Memory (MEM)

The Memory Stage operates as a side module of the Execute Stage and contains:

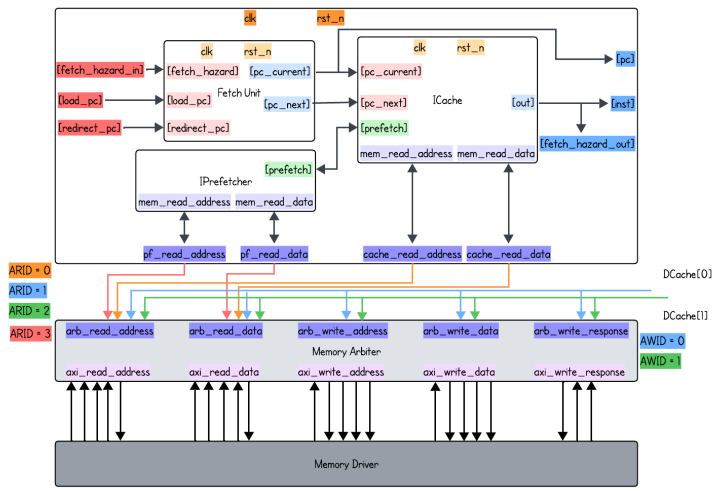
1. Load Queue (LQ): A circular buffer that holds pending load requests. Loads are pushed when the execute stage sends a memory request, and they are served to the D-Cache in ROB-index order. When a load's data returns (from cache hit, store-queue forwarding, or MSHR completion), it is sent back to the Execute Stage as a `mem_resp`.
2. Store Queue (SQ): A FIFO that buffers committed stores. Stores enter the SQ at commit time and are drained to the D-Cache when no pending loads are ahead. Store-to-load forwarding is implemented: before issuing a load to the cache, the SQ is checked for a matching address; on a hit, the store data is forwarded directly.

2.2.7 Commit (COM)

The Reorder Buffer is a circular array indexed by `RobIdx` (with an extra MSB phase bit). Each cycle, the Retire Unit inspects the two entries at the ROB head:

- If slot 0 is completed and not a mispredict, it is retired: the committed map is updated, the old physical register is freed, and the head pointer advances.
- Slot 1 can retire in the same cycle only if slot 0 also retires, and slot 1 is not a terminal instruction (MTC0 DONE).

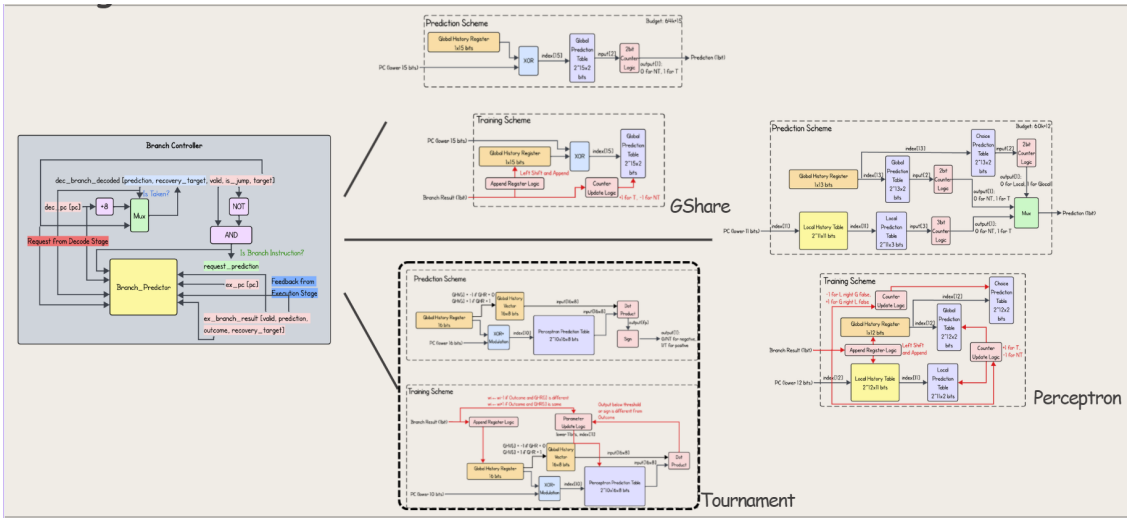
2.3 Instruction Cache Optimization: Stream Buffer Prefetcher



To reduce instruction-cache miss penalties, we implemented a **16-entry sequential stream buffer** that prefetches cache lines ahead of the current PC.

1. The stream buffer is a circular FIFO, where each entry stores a full cache-line address and `LINE_SIZE` words of instruction data. It operates on a dedicated AXI read port (ARID 3), independent of the I-Cache's own refill port, enabling concurrent fetches. A three-state FSM (`PF_IDLE`, `PF_REQUEST`, `PF_FILL`) drives the AXI handshake and word-by-word fill.
2. On each cycle, the I-Cache has two hit paths: (1) *internal hit*—the cache arrays contain the requested line, which is served directly with LRU update; (2) *external hit*—the stream buffer's head entry matches the requested line address, and the entire line is simultaneously installed into the cache (via the victim way selected by LRU) and served to the pipeline with zero extra latency. On a true miss (neither path hits), the I-Cache issues its own AXI refill while the prefetcher continues filling the buffer in parallel.When a branch redirect occurs, the stream buffer is reallocated: all valid entries are invalidated, and the `next_line` pointer is reset to the line following the new target. Any in-flight AXI fill is marked as `discard` so that its data is silently dropped.

2.4 Branch Predictor Design



We implemented and evaluated three branch predictors, all trained at commit time to avoid speculative pollution:

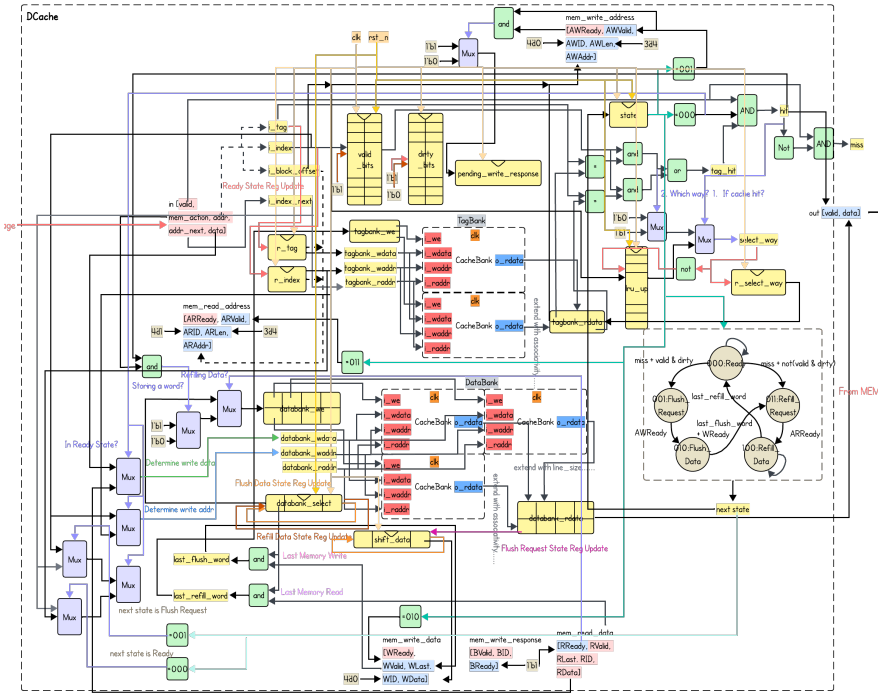
1. **GShare** XORs the lower bits of the PC with a Global History Register to index into a table of 2-bit saturating counters. The design space consists of two parameters: the number of Global History bits and the PHT index bits (constrained to be equal).

2. **Tournament** combines a global predictor indexed by a Global History Register, a local predictor fed by a local history table, and a 2-bit choice table that dynamically selects between local and global predictions. The design space is parameterized by five values: Local History bits, Global History bits, Meta Selection bits, Local PHT index bits, and Global PHT index bits. Training updates all three tables based on the committed branch outcome.

3. **Perceptron** maintains a Global History Vector and a weight table. Prediction is the sign of the dot product between the history vector and the weight row indexed by the PC. The design space is parameterized by the number of History bits, Perceptron index bits, and Weight bits. While theoretically capable of learning complex non-linear correlations, the perceptron's hardware cost (multipliers, wide adders) yielded only marginal accuracy improvements over the tournament in our experiments.

There are a large design space for which branch predictor to choose and which parameters to use. We performed a non-comprehensive sweep and settled on the **Tournament** predictor with design-space configuration `(10, 12, 12, 10, 12)`, corresponding to a 2^{10}-entry local history table, a 2^{12}-entry local predictor, a 2^{12}-entry global predictor, a 2^{10}-entry choice table, and a 12-bit Global History Register.

2.5 Data Cache Optimization: Lock-Up Free Cache



To fully exploit and speed-up potential of OoO processor, we pursued a dual-ported D-Cache design through **complete array duplication** to support lock-up-free.

1. The tag banks, data banks, valid bits, dirty bits, and pseudo-LRU state are fully duplicated. The primary port serves in-order loads (from the Load Queue head), store drains (from the Store Queue), and dirty write-backs. The secondary port is intended to serve speculative loads concurrently while the primary port is busy with a miss.

2. To maintain consistency between the two copies: (1) if the primary port writes a line dirty (store drain), it invalidates the corresponding line in the secondary copy (write-invalidate); (2) if the secondary port misses on a line that the primary holds dirty, a snoop-override path directly transfers the line from primary arrays to secondary in one cycle, avoiding an AXI round-trip. Two FSM states (`S_SNOOP_WAIT`, `S_SNOOP_RESP`) coordinate this handshake.

While the cache hardware itself is functional, the Memory Stage's FSM and input multiplexer have not yet been updated to fully exploit the secondary port. Completing this integration remains the primary next step.

3 Evaluation

3.1 Methodology

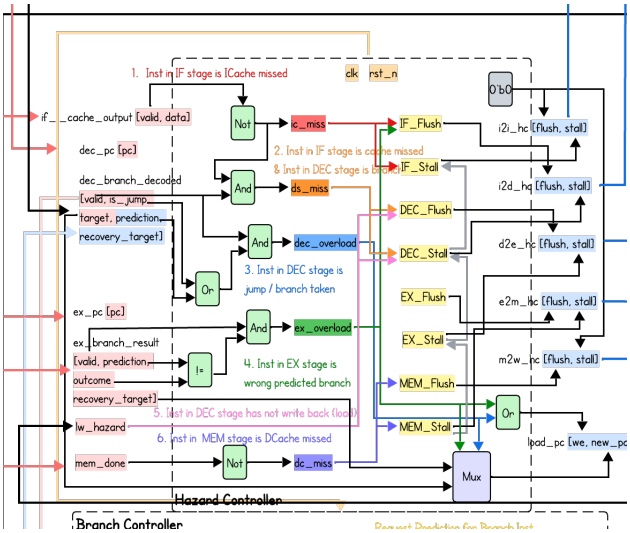


Figure 3: Centralized Hazard Unit in Decode Stage in Baseline Processor

The baseline processor uses a centralized hazard unit in the decode stage to detect all categories of hazards based on per-stage status signals such as ICache miss and DCache miss, enabling systematic collection of performance statistics and insight into processor behavior at runtime. We adopted the same philosophy for our optimized OoO design: a centralized hazard controller at the top level computes stall and flush signals for each of the seven pipeline stages. Each stage exposes a `hazard_out` signal bundle, and the top-level controller resolves priorities and propagates the appropriate control back to each stage via `hazard_in`.

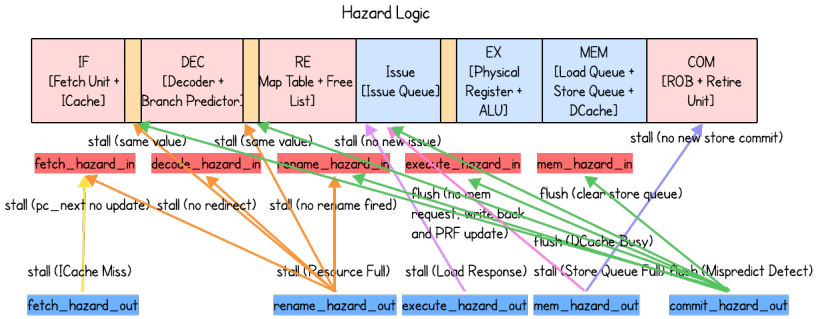


Figure 4: Centralized Hazard Unit in Top Level in Optimized Processor

The key hazard signals and their meanings are summarized below. Stalls propagate backward: a downstream stall freezes all upstream stage registers and pipeline registers. Flushes propagate forward: a commit-stage flush invalidates all in-flight instructions. The commit-stage flush carries the highest priority and overrides all other signals.

Stage	Signal	Meaning
Fetch	`fetch_hazard_out.stall`	I-cache miss (stall front end)
Fetch	`fetch_hazard_out.flush`	Decode redirect (branch target known early)
Rename	`rename_hazard_out.stall`	ROB full, IQ full, or free list empty
Execute	`execute_hazard_out.stall`	Load response returning (stall IQ issue)
Execute	`execute_hazard_out.flush`	Self-replay (load collides with its own response)
Memory	`mem_hazard_out.stall`	Store queue drain full
Memory	`mem_hazard_out.flush`	Load queue or MSHR full
Commit	`commit_hazard_out.flush`	Branch mispredict (full pipeline flush)

Table 2: Summary of hazard signals and their semantics.

3.2 Performance Results

We compare our final superscalar out-of-order processor (with all optimizations) against the in-order baseline processor. Four benchmarks are used: NQueens, Coin, QuickSort, and Esift2. All statistics are collected through the hardware performance counters that tracking cycle counts, commit counts, cache events, stall causes, and branch outcomes. Since the final processor integrates all design features simultaneously, the comparison does not isolate individual optimizations. Rather, it demonstrates the cumulative effect of the full design.

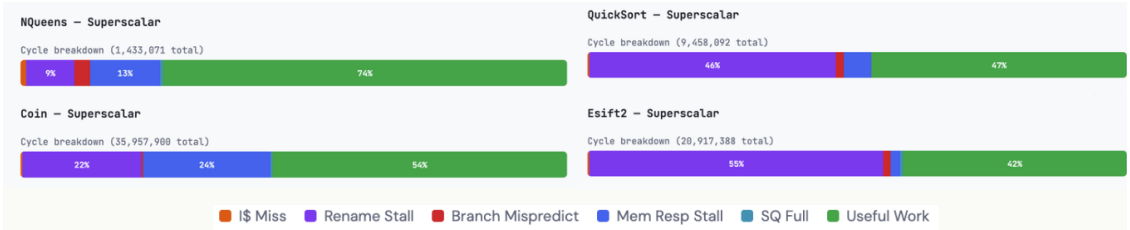
Metric	NQueens	Coin	QuickSort	Esift2	Geo. Mean
Superscalar CPI	1.41	1.01	2.30	2.66	—
Baseline CPI	13.63	1.20	2.37	2.80	—
Speedup	9.66×	1.19×	1.03×	1.05×	**1.88**×
#1 Bottleneck (Super)	Mem resp stall	Mem resp stall	Rename stall	Rename stall	—
#1 Bottleneck (Base)	I$ miss/flush	EX overload	D$ miss	D$ miss	—

Table 3: Performance comparison: optimized processor vs. in-order baseline.

All benchmarks show improvement over the baseline, with each benefiting from different optimizations throughout the design process. NQueens shows the largest speedup (9.66×) because the baseline was severely bottlenecked by instruction-cache misses, which our larger I-cache and stream buffer nearly eliminate. Coin achieves the highest absolute IPC (0.995) due to its compact loop structure that benefits heavily from the branch predictor and superscalar issue. QuickSort and Esift2 show modest gains; their performance is limited by rename stalls caused by a full Issue Queue, indicating that pending loads hold up dependent instructions.

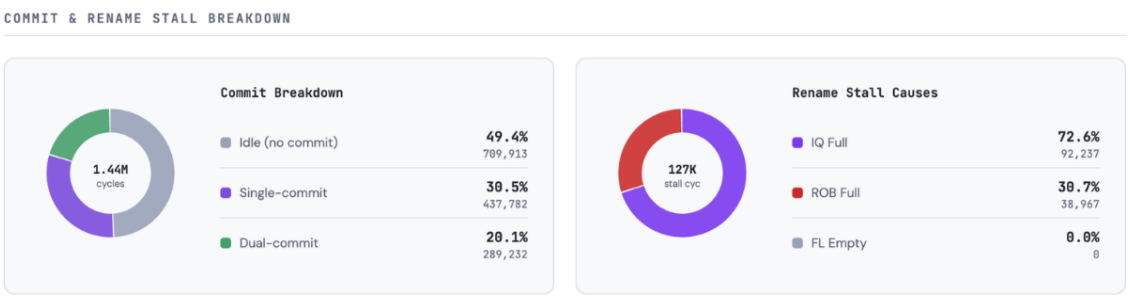
3.3 Fine-grained analysis

1. **Cycle Breakdown:**



The cycle breakdown reveals that NQueens and Coin spend more of their cycles on useful work compared to QuickSort and Esift2. For NQueens and Coin, the dominant remaining stalls are memory-response stalls and rename stalls, suggesting these benchmarks would benefit most from structural duplication in the execution stage (e.g., more ALU ports or wider issue). For QuickSort and Esift2, I-cache misses still dominate the stall breakdown, indicating further cache improvements could yield additional gains.

2. **Commit and Rename Stall:**



Fine-grained analysis of the NQueens benchmark reveals: The dominance of IQ-full stalls indicates that many instructions wait in the IQ for operands produced by long-latency loads. This points directly to the benefit of exploiting memory-level parallelism (MLP) via the MSHR file and lock-up-free cache integration.

3. **Cache Performance:**

The instruction cache achieves near-perfect hit rates after the initial compulsory misses and the data cache also achieves a 95.7% hit rate, with store-to-load forwarding accounting for 0.7% of accesses. Under the final processor configuration, NQueens no longer suffers significantly from cache hit-rate issues; however, cache *latency* remains a contributing factor to IQ-full stalls, as long-latency load misses keep dependent instructions waiting in the Issue Queue.

4 Conclusion and Future Work

We have designed and implemented a 2-way superscalar out-of-order MIPS processor that achieves a 1.88× geometric-mean speedup over the in-order baseline across four benchmarks, with a peak IPC of 0.995 on the Coin benchmark. The primary remaining bottleneck is IQ-full stalls caused by long-latency loads blocking dependent instructions. The rename-stall analysis confirms that 72.6% of stalls originate from a full Issue Queue, largely attributable to cache-miss latency rather than hit-rate deficiency. Our next steps include:

1. **Lock-up-free D-Cache integration:** completing the Memory Stage FSM and input-multiplexer updates to fully exploit the dual-ported D-Cache's secondary port, enabling memory-level parallelism and reducing IQ pressure.
2. **Expanding IQ and ROB sizes:** increasing the instruction window to tolerate more in-flight instructions waiting for long-latency operations.
3. **Less conservative superscalar issue:** relaxing the single-issue fallback rules (e.g., allowing slot 1 to issue memory instructions) to improve IPC on benchmarks with high memory traffic.
4. **Branch prediction tuning:** per-benchmark profiling of the tournament predictor parameters to further reduce misprediction penalties.
5. **Step-by-step performance tracking:** adding a per-commit performance comparison across git commits to isolate the contribution of each individual optimization.